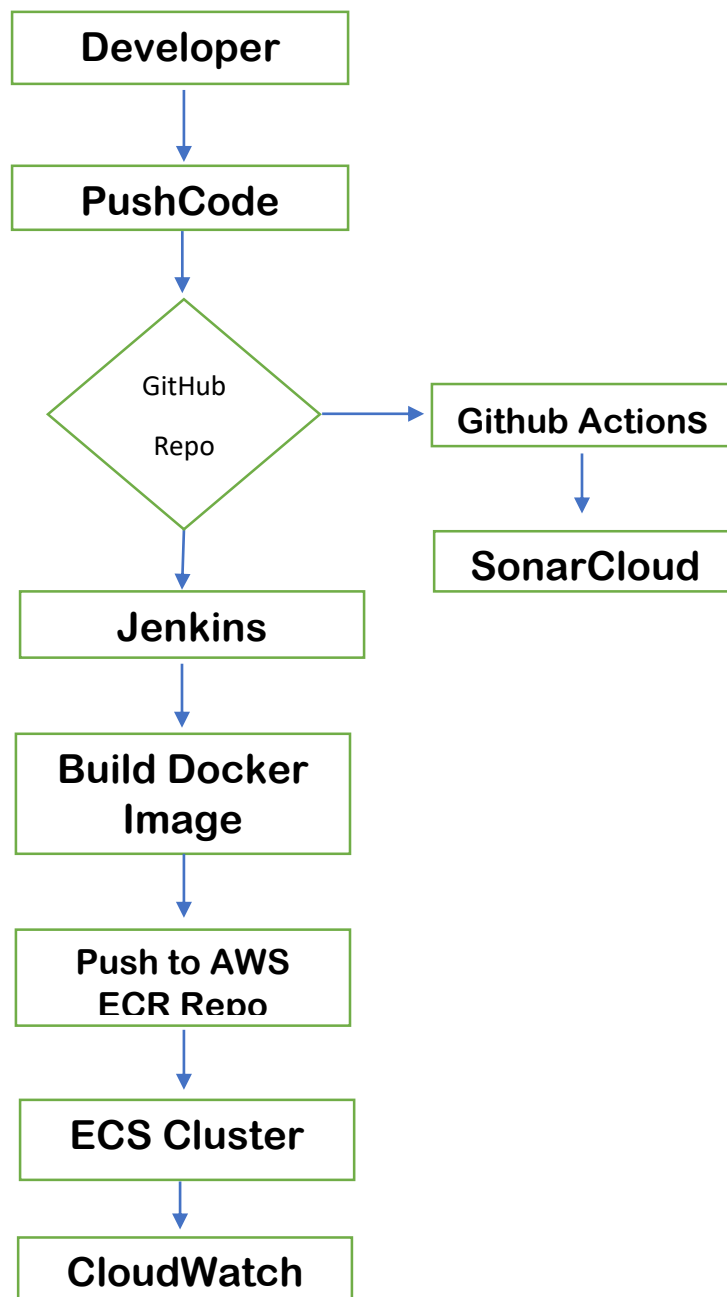


Github Link: <https://github.com/kumarsuresh03/Backend.git>

Problem Statement:

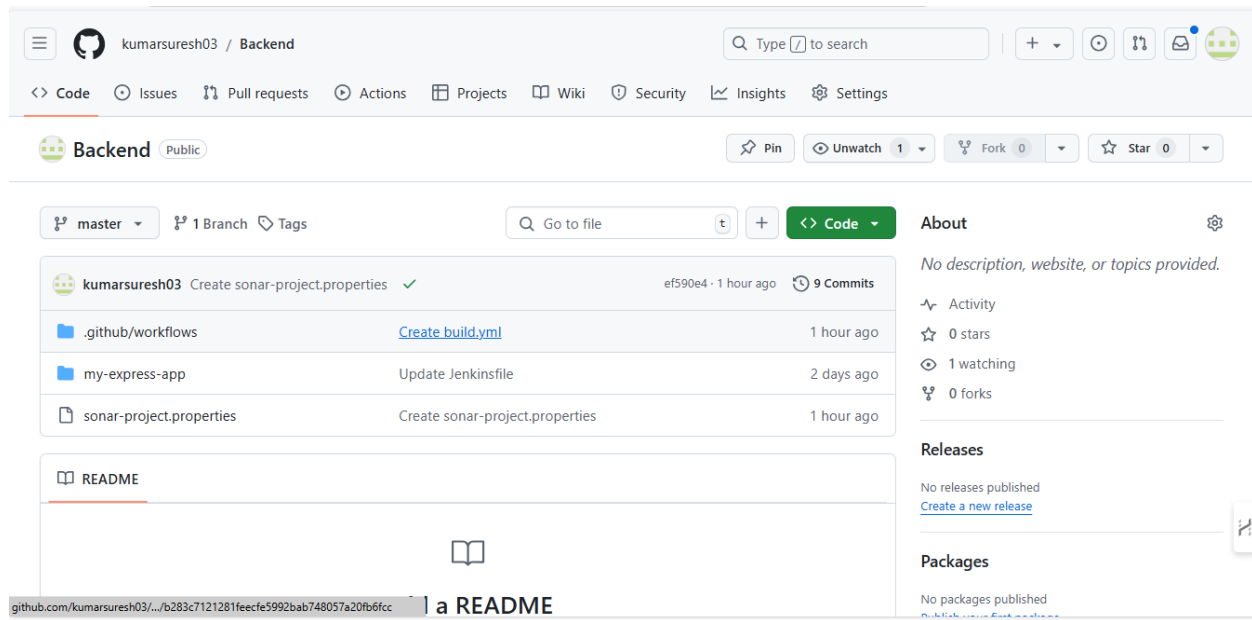
1. Choose any backend application framework
2. Design a CI/CD pipeline blueprint for your chosen framework, demonstrating how it would support a seamless development-to-deployment workflow



Created Backend Application using Express and node.js and Push GitHub repo

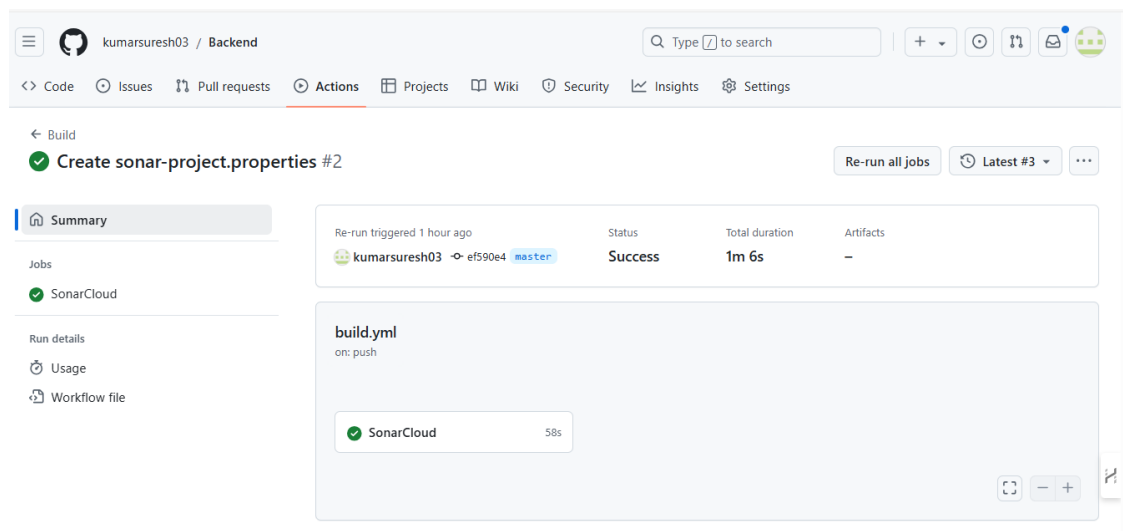
1.Source Stage – GitHub

- Developers push code to a GitHub repository

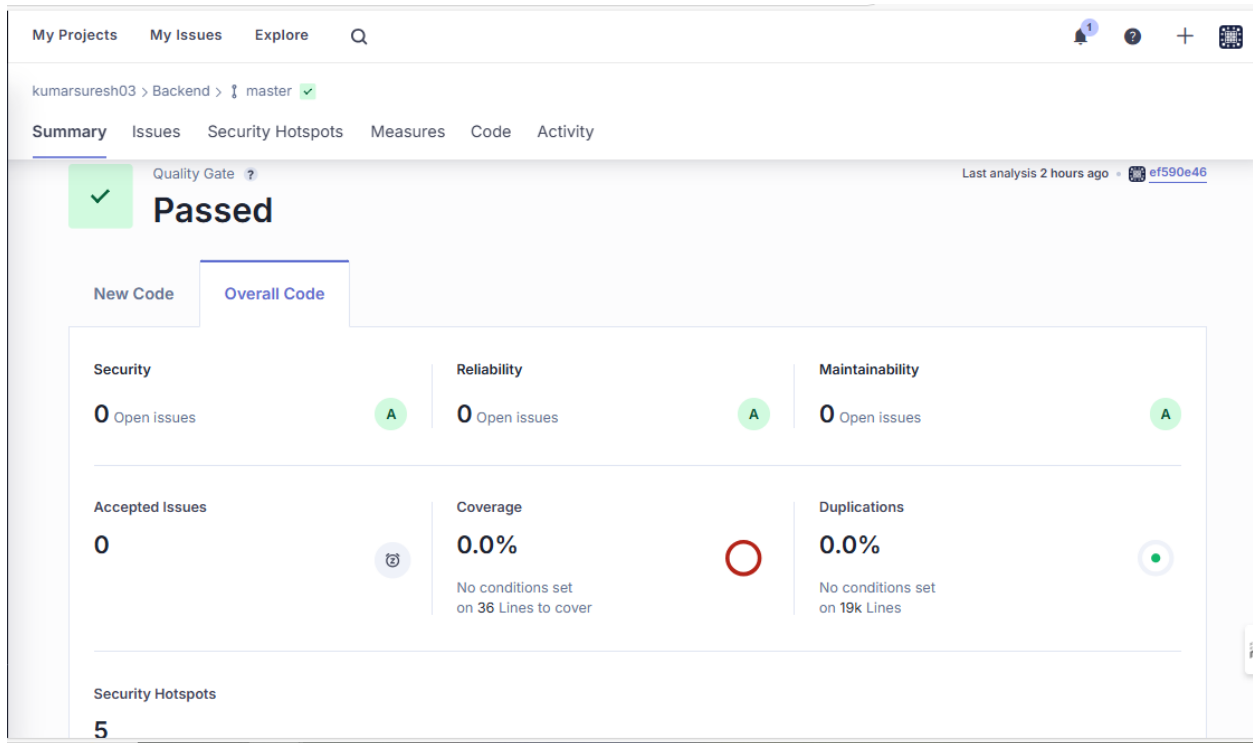


2. Continuous Integration - GitHub Actions & SonarCloud

We have created workflow github actions to sonarcloud



SonarCloud analyzing the code for quality and security vulnerabilities.



3. Continuous Delivery - Jenkins to AWS ECR

Build Trigger:

- Set up a webhook or configure Jenkins to poll the GitHub repository.

Webhooks / Add webhook

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [our developer documentation](#).

Payload URL *

Content type *

Secret

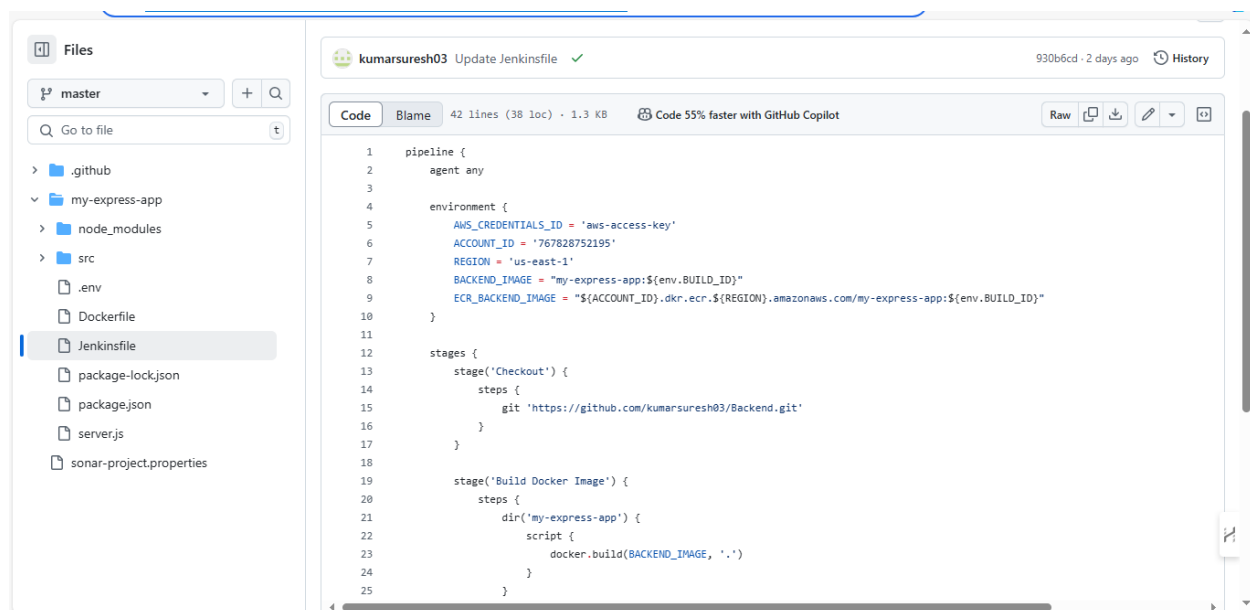
SSL verification
☐ By default, we verify SSL certificates when delivering payloads.

☒ **Enable SSL verification** ☐ **Disable (not recommended)**

Docker Build and Push to ECR:

Jenkins file path: [Backend/my-express-app/Jenkinsfile at master · kumarsuresh03/Backend](#)

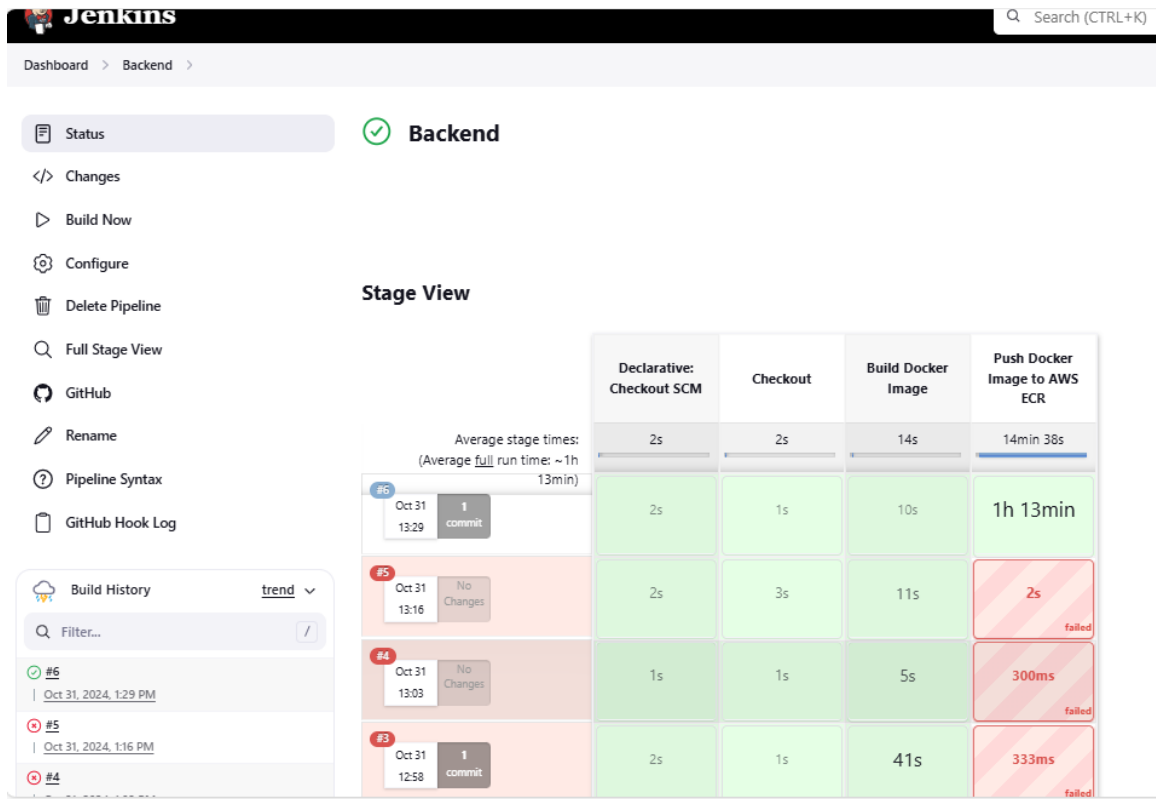
We have configured the jenkins file in Github repo



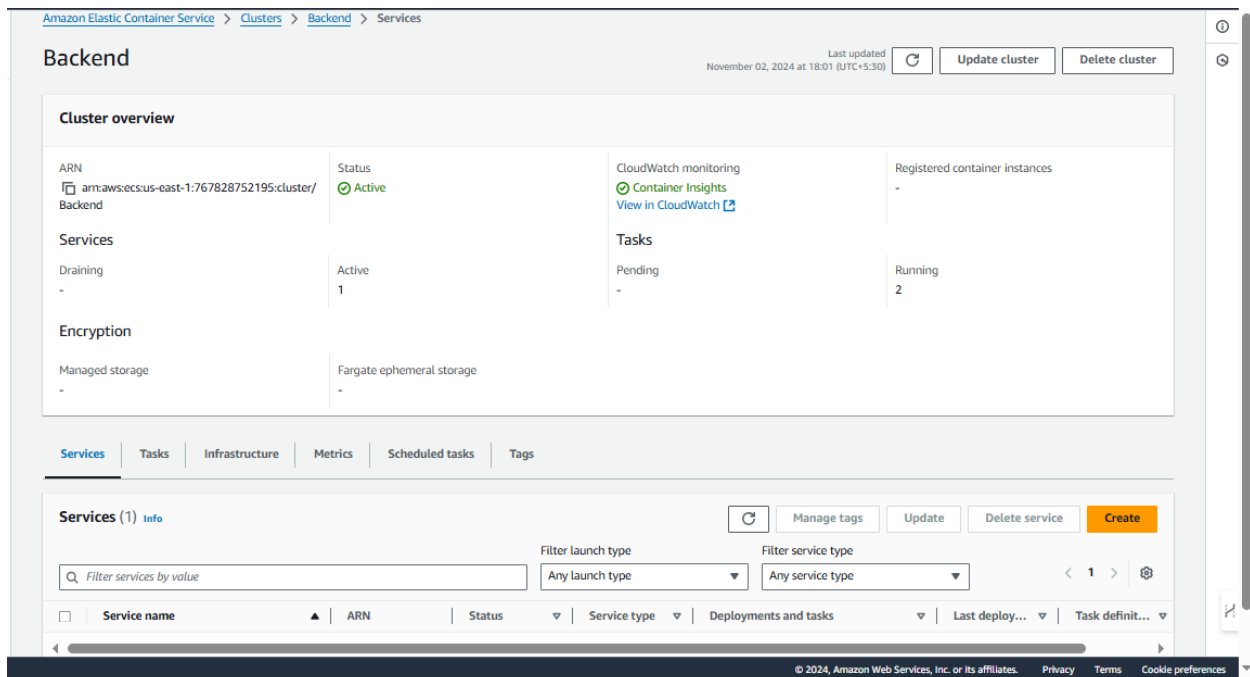
The screenshot shows a GitHub repository interface. On the left, the file explorer shows the directory structure: `my-express-app` > `src` > `Jenkinsfile`. The main area displays the content of the `Jenkinsfile`, which is a Jenkins pipeline script. The script defines a pipeline with an agent, environment variables for AWS credentials and ECR, and two stages: 'Checkout' and 'Build Docker Image'.

```
1 pipeline {
2   agent any
3
4   environment {
5     AWS_CREDENTIALS_ID = 'aws-access-key'
6     ACCOUNT_ID = '767828752195'
7     REGION = 'us-east-1'
8     BACKEND_IMAGE = "my-express-app:${env.BUILD_ID}"
9     ECR_BACKEND_IMAGE = "${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com/my-express-app:${env.BUILD_ID}"
10  }
11
12  stages {
13    stage('Checkout') {
14      steps {
15        git 'https://github.com/kumarsuresh03/Backend.git'
16      }
17    }
18
19    stage('Build Docker Image') {
20      steps {
21        dir('my-express-app') {
22          script {
23            docker.build(BACKEND_IMAGE, '.')
24          }
25        }
26      }
27    }
28  }
29 }
```

Jenkins builds the Docker image for the Express application and pushes it to AWS ECR



4.Create an ECS Cluster



For this cluster we have created one task and deployed one service

In this task running the container

The screenshot shows the Amazon Elastic Container Service (ECS) console. The breadcrumb navigation is: Amazon Elastic Container Service > Clusters > Backend > Tasks > 6e29e6c504b14734a76ea2254363e5da > Configuration. The task ID is 6e29e6c504b14734a76ea2254363e5da. The last updated time is November 02, 2024 at 18:03 (UTC+5:30). The task is in the 'Running' state. The Fargate ephemeral storage is 20 GiB. The configuration section is expanded, showing details for the 'backend' container. The container details include: Image URI, Essential, Command, Log configuration, Restart policy, Network bindings, Docker labels and hosts, Environment variables and files, and Volume configuration.

Amazon Elastic Container Service > Clusters > Backend > Tasks > 6e29e6c504b14734a76ea2254363e5da > Configuration

6e29e6c504b14734a76ea2254363e5da

Last updated November 02, 2024 at 18:03 (UTC+5:30) [Refresh] [Stop]

Configuration | Logs | Networking | Volumes (0) | Tags

Task overview

ARN arn:aws:ecs:us-east-1:767828752195:task/Backend/6e29e6c504b14734a76ea2254363e5da	Last status Running	Desired status Running	Started/Created at November 02, 2024 at 13:40 (UTC+5:30) November 02, 2024 at 13:40 (UTC+5:30)
---	------------------------	---------------------------	--

Fargate ephemeral storage

Encryption Default AWS Fargate encryption	Size (GiB) 20
--	------------------

Configuration

Container details for backend

Details | Log configuration | Restart policy | Network bindings | Docker labels and hosts | Environment variables and files | Volume configuration

Details

Image URI	Essential	Command
-----------	-----------	---------

After we need to deploy the service and added load balancer in this service

The screenshot shows the Amazon Elastic Container Service (ECS) console. The breadcrumb navigation is: Amazon Elastic Container Service > Clusters > Backend > Services > Backend > Health. The service name is Backend. The last updated time is November 02, 2024 at 18:04 (UTC+5:30). The service is in the 'Active' state. The status bar shows 1 Pending and 1 Running task. The load balancer health is shown as (Application Load Balancer) Backen-ALB. The listener protocol/port is HTTP-3000. The target group name/protocol is backend:HTTP. The health check path is /. The targets are 1 Healthy and 0 Unhealthy.

Amazon Elastic Container Service > Clusters > Backend > Services > Backend > Health

Backend

Last updated November 02, 2024 at 18:04 (UTC+5:30) [Refresh] [Update service] [Delete service]

Health and metrics | Tasks | Logs | Deployments | Events | Configuration and networking | Tags

Status

ARN arn:aws:ecs:us-east-1:767828752195:service/Backend/Backend	Status Active	Tasks (1 Desired) 0 Pending 1 Running
---	------------------	--

Deployments current state
1 Completed

Health check grace period
0 seconds

Load balancer health

(Application Load Balancer) Backen-ALB

View load balancer

Listener protocol/port HTTP-3000	Target group name/protocol backend:HTTP	Health check path /	Targets (1 total) 1 Healthy 0 Unhealthy
-------------------------------------	--	------------------------	--

View Container Insights

And we used the Application Load Balancer for this application

EC2 > Load balancers > Backen-ALB

Backen-ALB

Details

Load balancer type Application	Status Active	VPC vpc-022ef073ddb81c307	Load balancer IP address type IPv4
Scheme Internet-facing	Hosted zone Z35SXDOTRQ7X7K	Availability Zones subnet-0bf350422143f9c95 us-east-1b (use1-az4) subnet-0ea8af1568e146863 us-east-1a (use1-az2)	Date created November 2, 2024, 13:25 (UTC+05:30)
Load balancer ARN arn:aws:elasticloadbalancing:us-east-1:767828752195:loadbalancer/app/Backen-ALB/8d39d1b173c0776e		DNS name Backen-ALB-1302253125.us-east-1.elb.amazonaws.com (A Record)	

Listeners and rules | Network mapping | Resource map - new | Security | Monitoring | Integrations | Attributes | Tags

Listeners and rules (1) info

A listener checks for connection requests on its configured protocol and port. Traffic received by the listener is routed according to the default action and any additional rules.

Filter listeners

Protocol:Port	Default action	Rules	ARN	Security policy	Default SSL/TLS certificate	mTLS
---------------	----------------	-------	-----	-----------------	-----------------------------	------

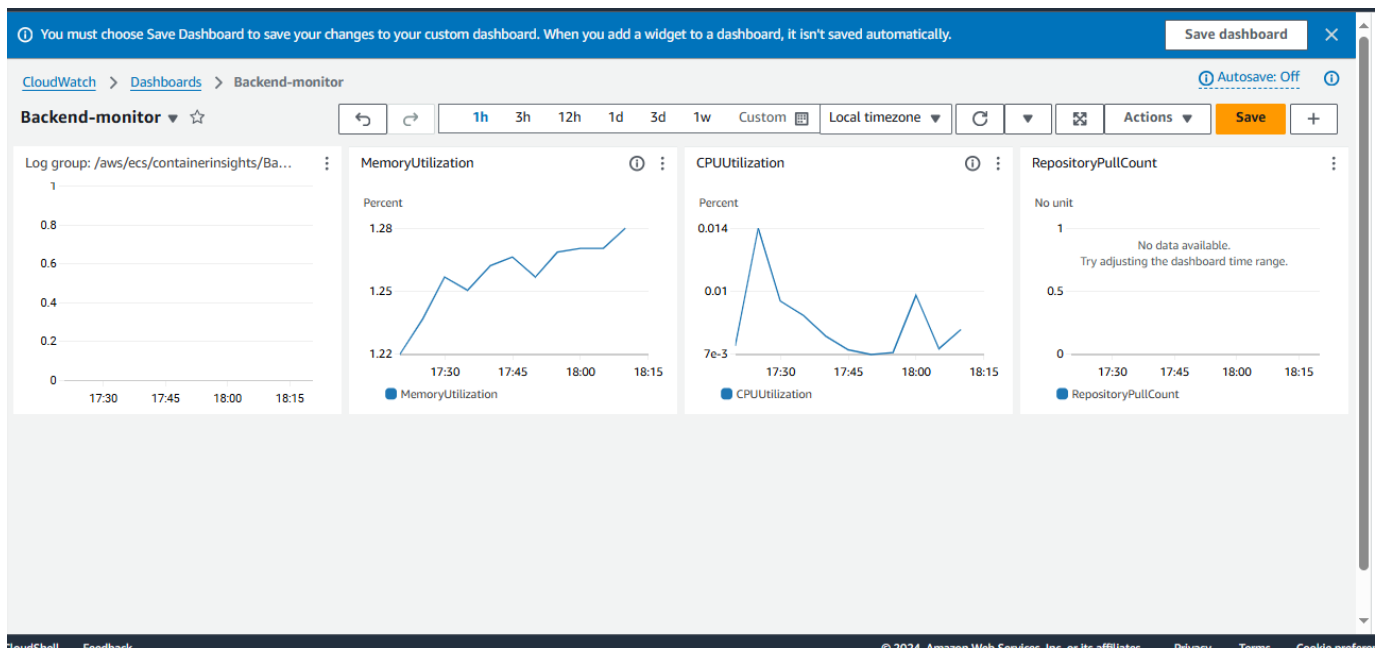
Successfully running the container the ip is [54.227.106.163:3000](#)



5. Monitoring – CloudWatch

Once the application is deployed on ECS, set up CloudWatch for monitoring the application's health, performance, and logs.

Set Up Dashboards: Use CloudWatch Dashboards to get a consolidated view of these metrics.



Create Alarms: Set up alarms in CloudWatch to monitor key metrics for CPU and memory utilization

